LAPORAN PRAKTIKUM PERTEMUAN 3

PEMROGRAMAN MOBILE



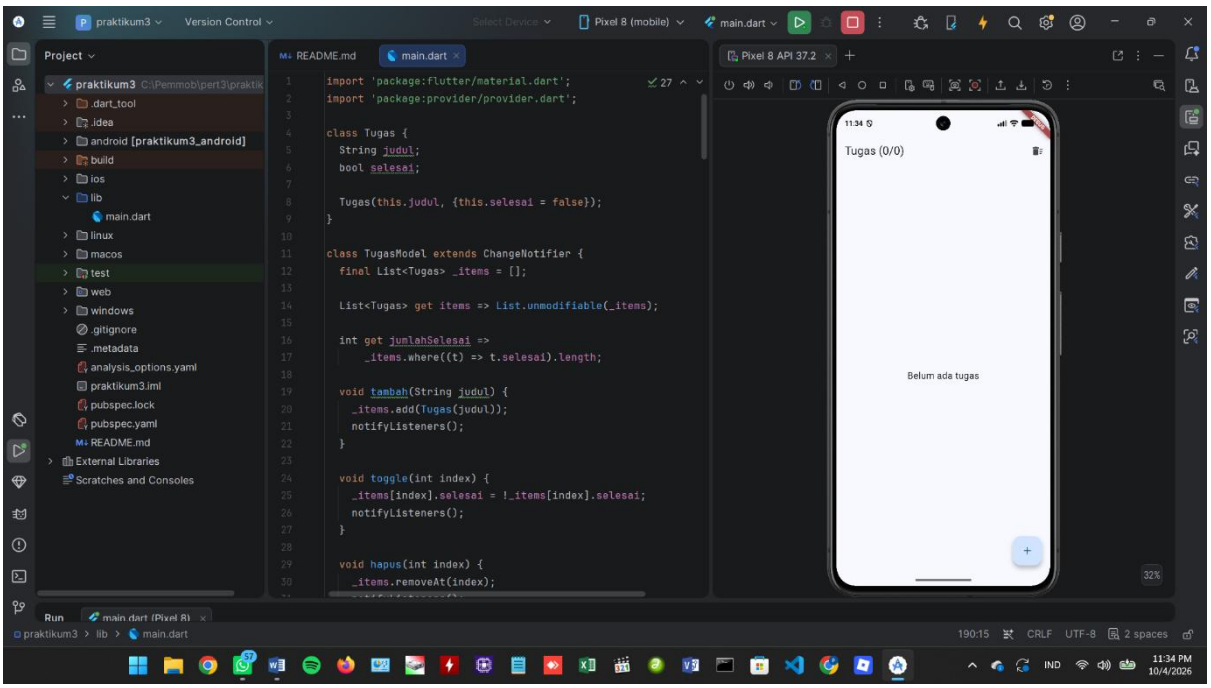
Disusun Oleh :
Lingga Naura Rediana (20240801055)

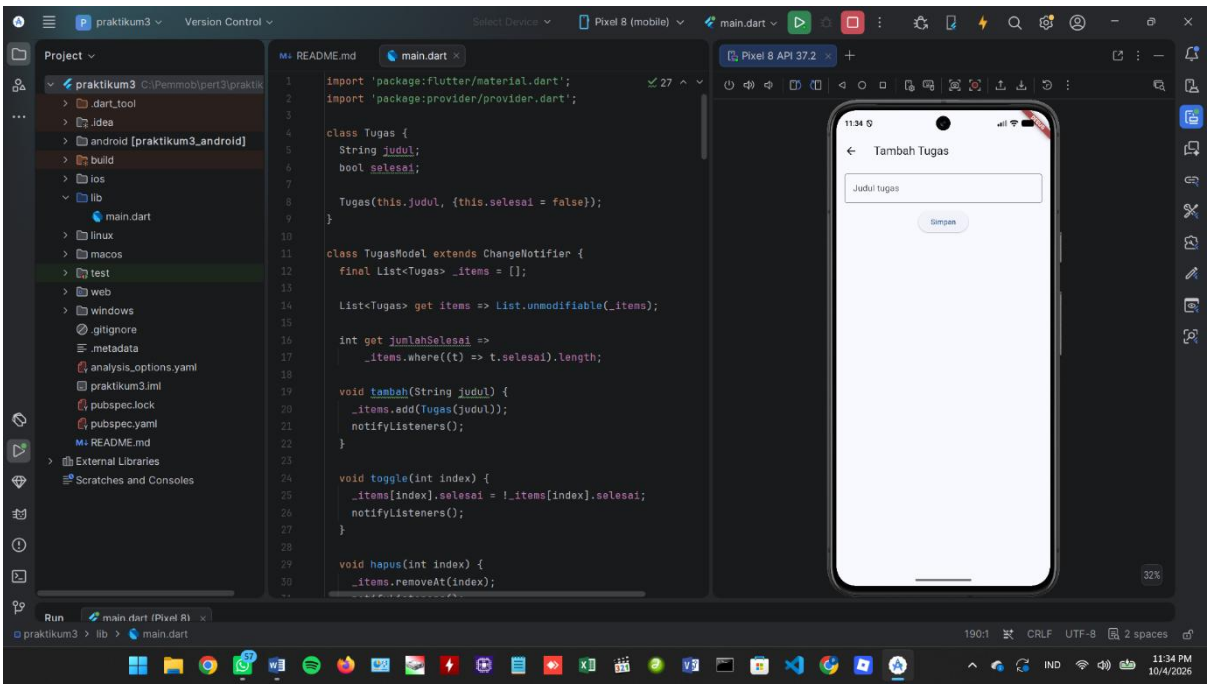
Dosen Pengampu :
JEFRY SUNUPURWA ASRI , S.Kom., M.Kom

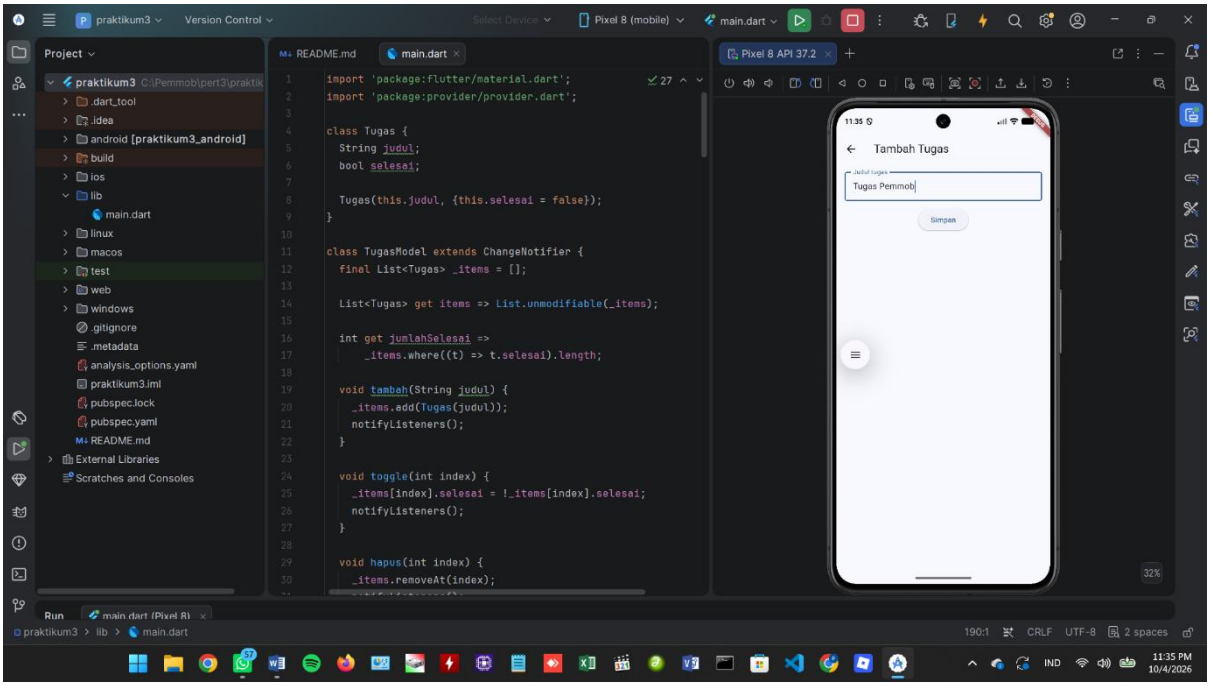
Mata Kuliah :
Pemrograman Mobile
CR001

FAKULTAS ILMU KOMPUTER
UNIVERSITAS ESA UNGGUL
2026

Tampilan Praktikum 3 & Latihan Mandiri








```dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

class Tugas {
  String judul;
  bool selesai;

  Tugas(this.judul, {this.selesai = false});
}

class TugasModel extends ChangeNotifier {
  final List<Tugas> _items = [];

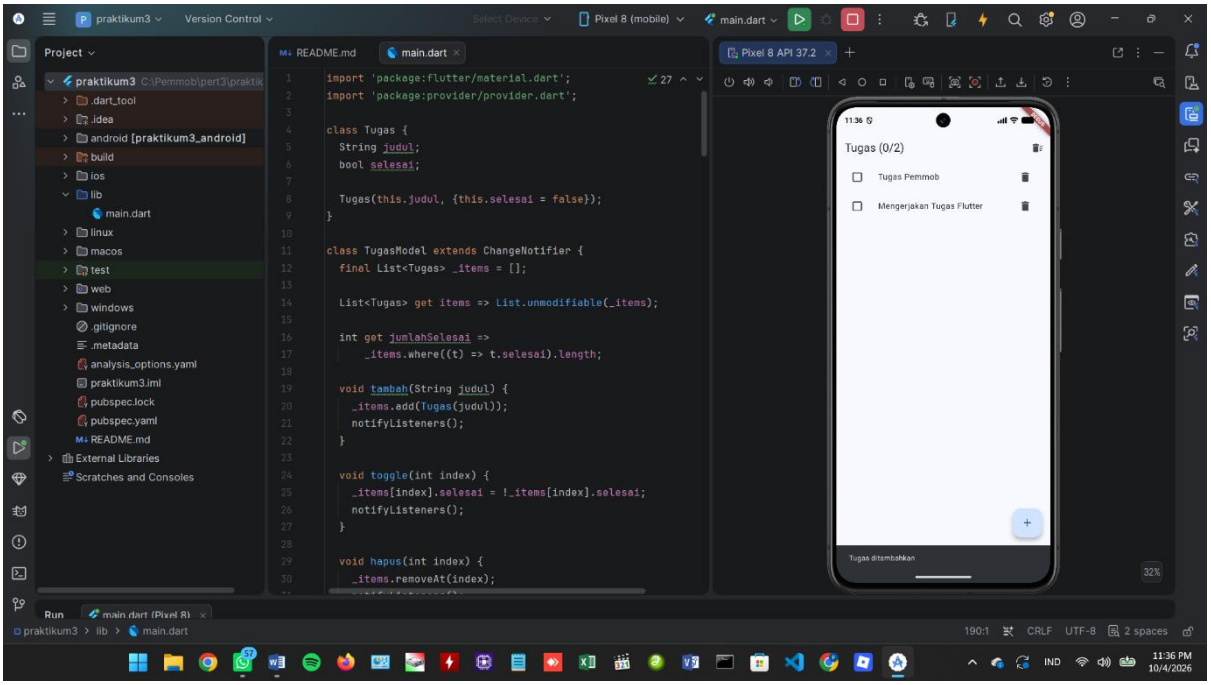
  List<Tugas> get items => List.unmodifiable(_items);

  int get jumlahSelesai =>
      _items.where((t) => t.selesai).length;

  void tambah(String judul) {
    _items.add(Tugas(judul));
    notifyListeners();
  }

  void toggle(int index) {
    _items[index].selesai = !_items[index].selesai;
    notifyListeners();
  }

  void hapus(int index) {
    _items.removeAt(index);
```




```dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

class Tugas {
  String judul;
  bool selesai;

  Tugas(this.judul, {this.selesai = false});
}

class TugasModel extends ChangeNotifier {
  final List<Tugas> _items = [];

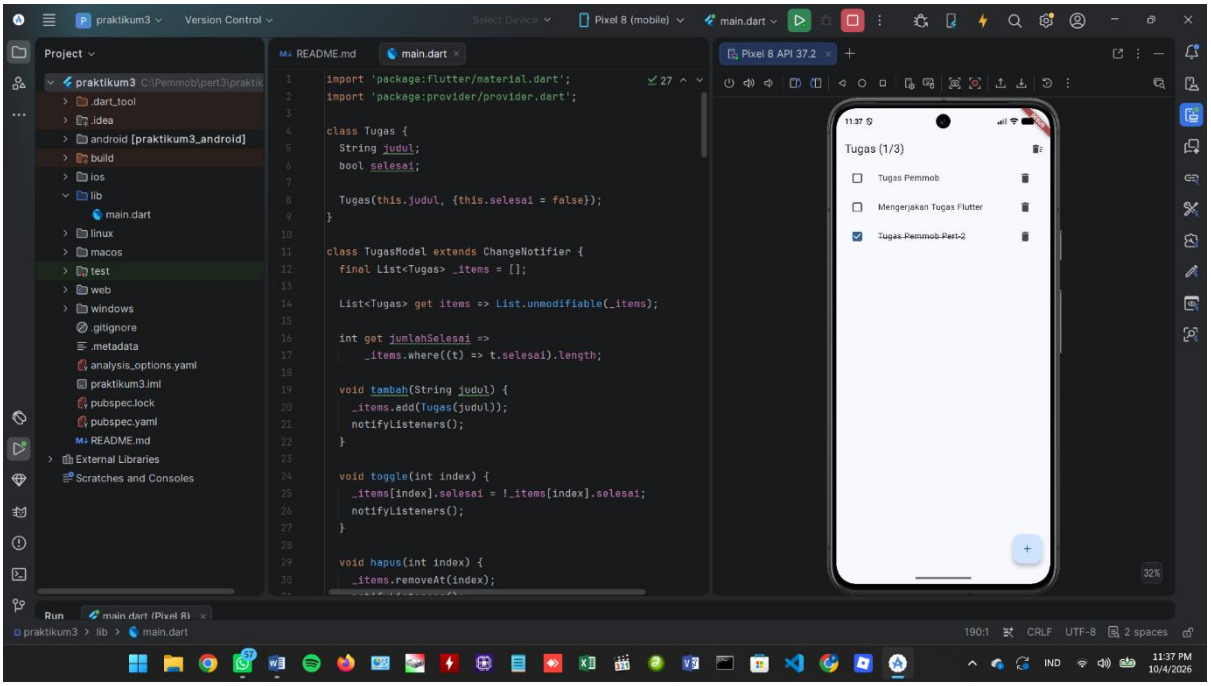
  List<Tugas> get items => List.unmodifiable(_items);

  int get jumlahSelesai =>
      _items.where((t) => t.selesai).length;

  void tambah(String judul) {
    _items.add(Tugas(judul));
    notifyListeners();
  }

  void toggle(int index) {
    _items[index].selesai = !_items[index].selesai;
    notifyListeners();
  }

  void hapus(int index) {
    _items.removeAt(index);
```




```dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

class Tugas {
  String judul;
  bool selesai;

  Tugas(this.judul, {this.selesai = false});
}

class TugasModel extends ChangeNotifier {
  final List<Tugas> _items = [];

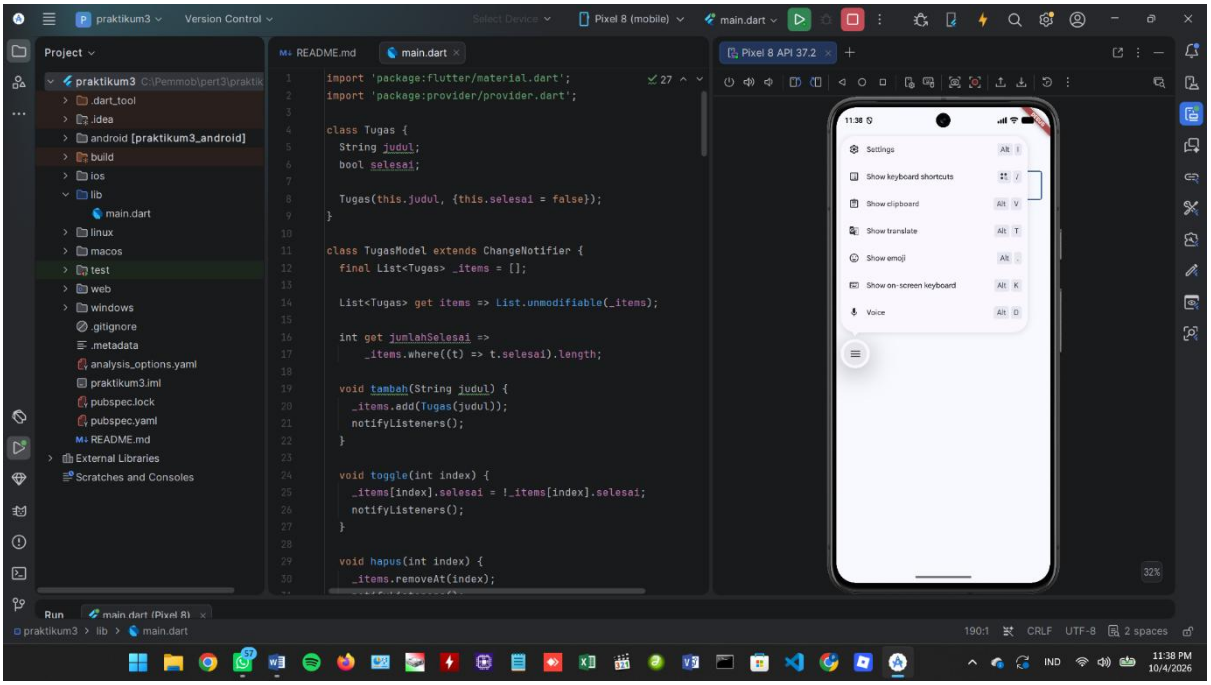
  List<Tugas> get items => List.unmodifiable(_items);

  int get jumlahSelesai =>
      _items.where((t) => t.selesai).length;

  void tambah(String judul) {
    _items.add(Tugas(judul));
    notifyListeners();
  }

  void toggle(int index) {
    _items[index].selesai = !_items[index].selesai;
    notifyListeners();
  }

  void hapus(int index) {
    _items.removeAt(index);
```




```dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

class Tugas {
  String judul;
  bool selesai;

  Tugas(this.judul, {this.selesai = false});
}

class TugasModel extends ChangeNotifier {
  final List<Tugas> _items = [];

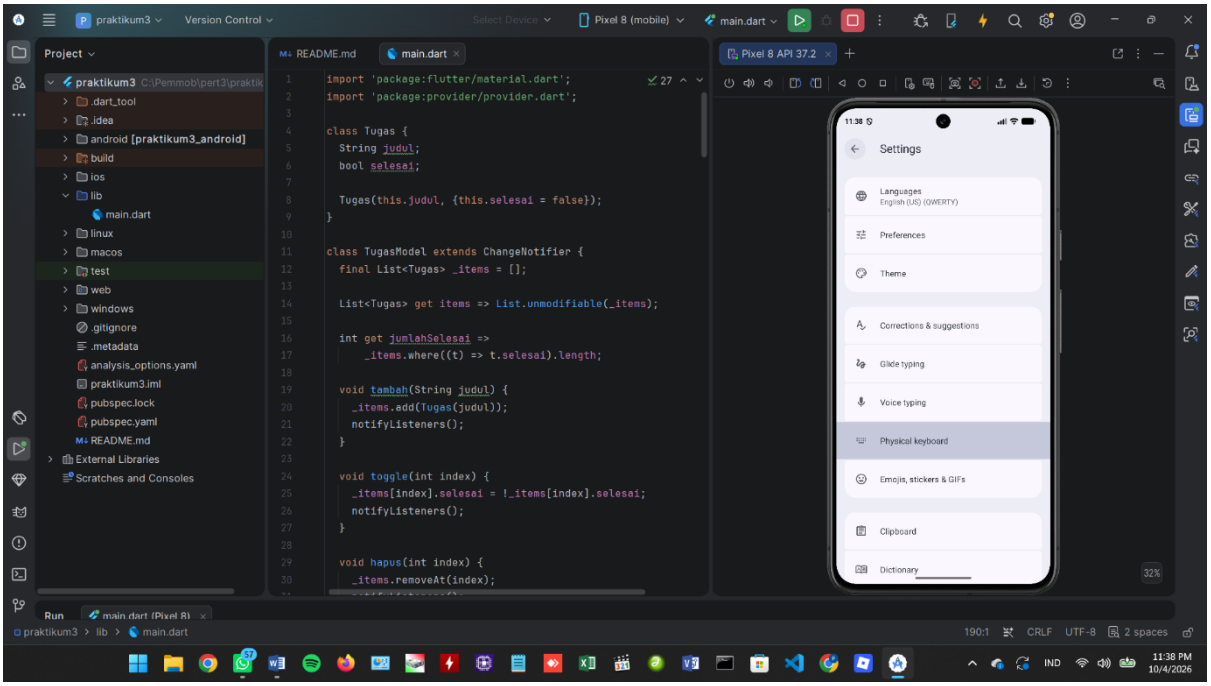
  List<Tugas> get items => List.unmodifiable(_items);

  int get jumlahSelesai =>
      _items.where((t) => t.selesai).length;

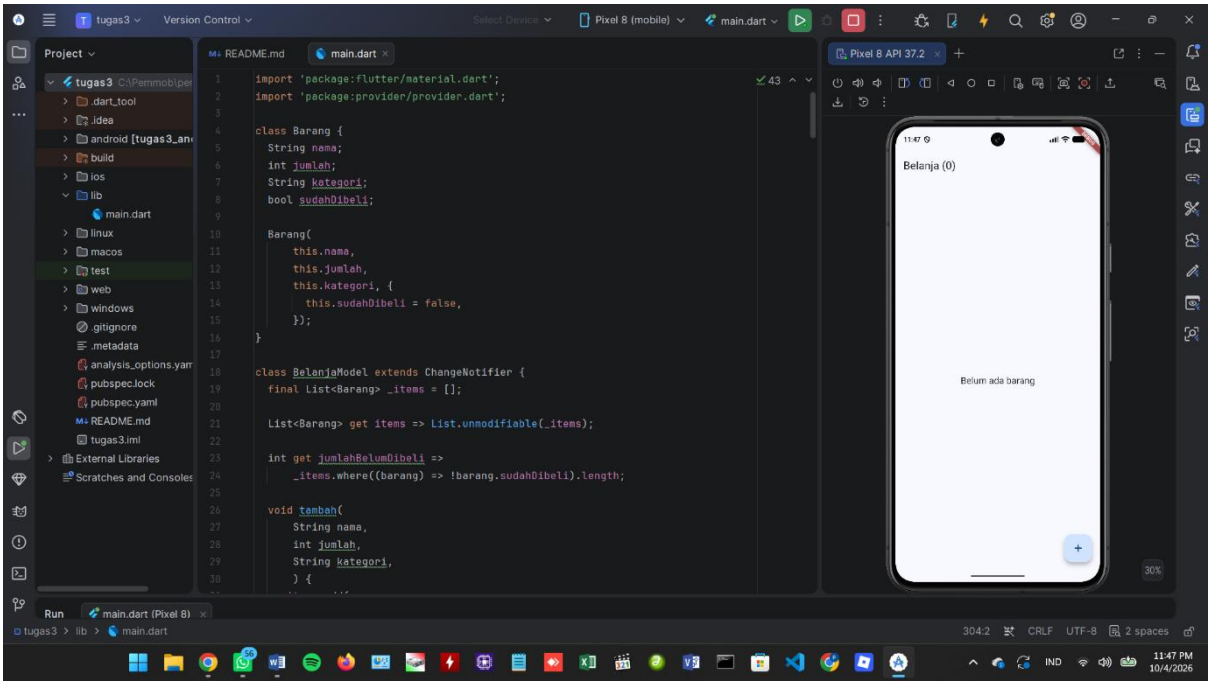
  void tambah(String judul) {
    _items.add(Tugas(judul));
    notifyListeners();
  }

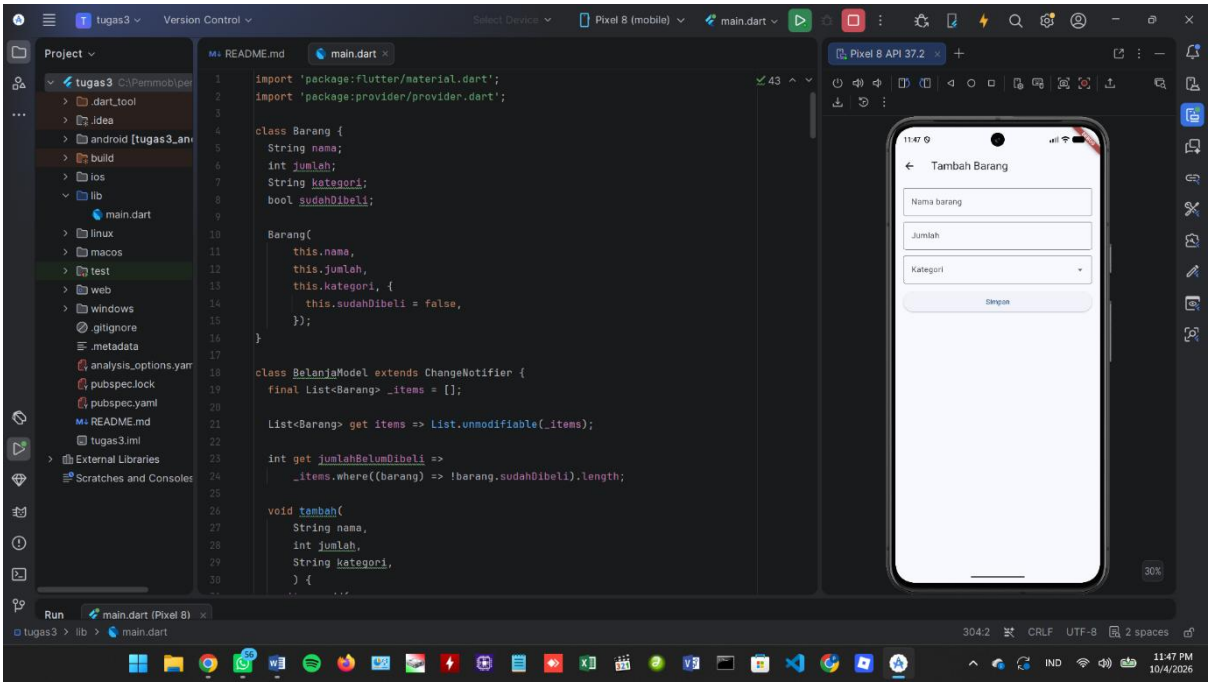
  void toggle(int index) {
    _items[index].selesai = !_items[index].selesai;
    notifyListeners();
  }

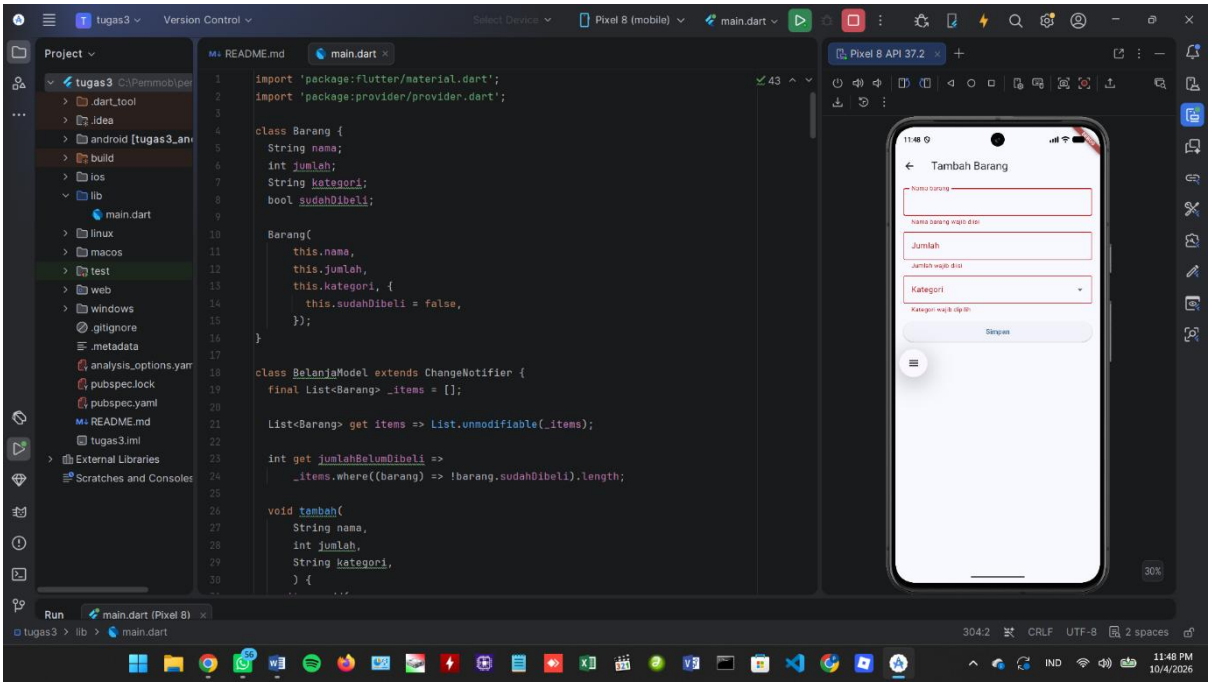
  void hapus(int index) {
    _items.removeAt(index);
```

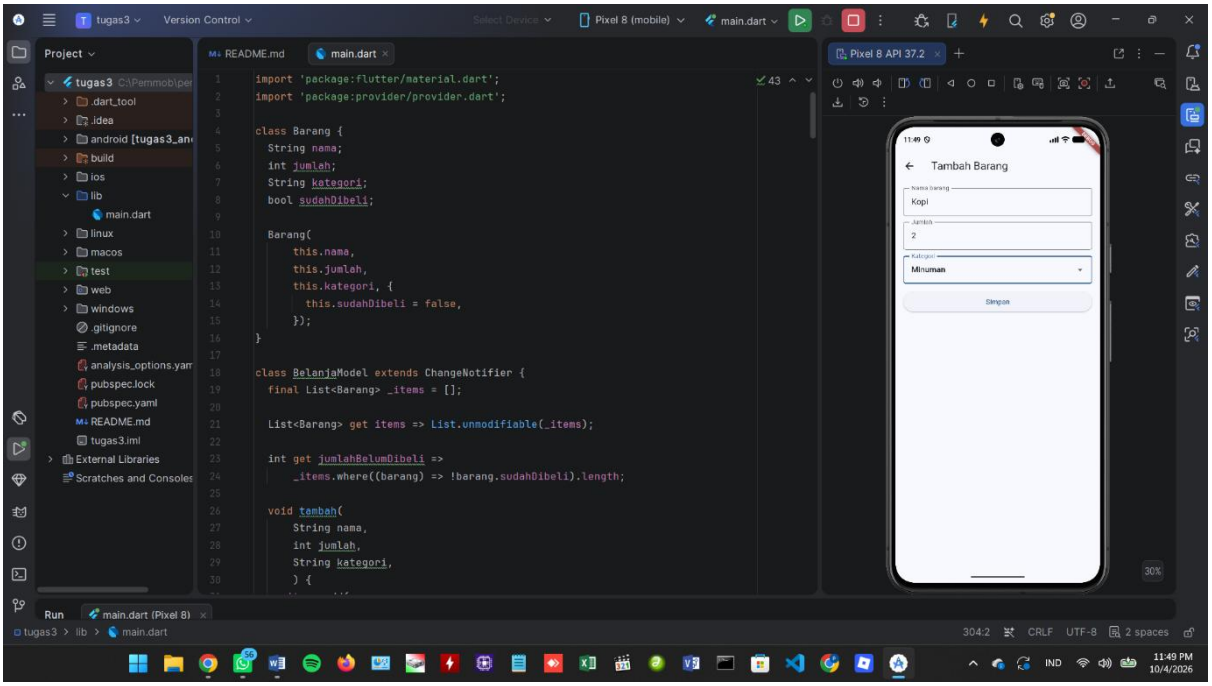


Tampilan Tugas Mandiri




Tambah Barang
Nama barang
Jumlah
Kategori
Simpan


Tambah Barang
Nama barang
Jumlah
Kategori
Simpan




```dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

class Barang {
  String nama;
  int jumlah;
  String kategori;
  bool sudahDibeli;

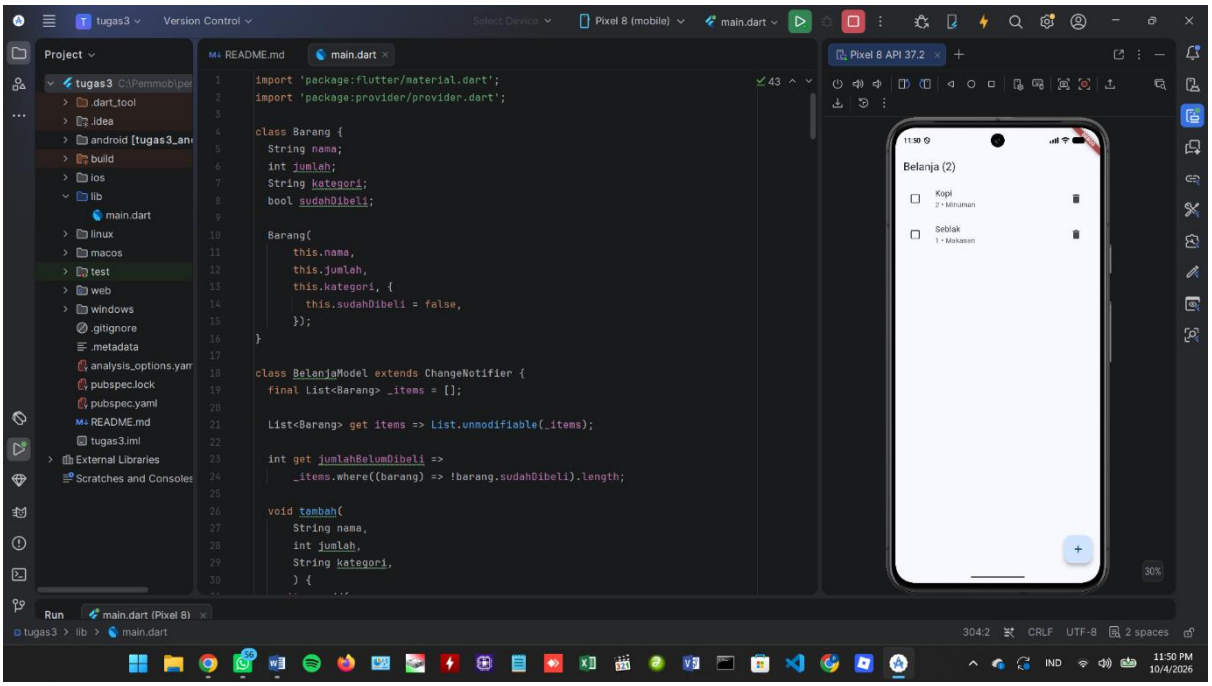
  Barang(
      this.nama,
      this.jumlah,
      this.kategori, {
        this.sudahDibeli = false,
      });
}

class BelanjaModel extends ChangeNotifier {
  final List<Barang> _items = [];

  List<Barang> get items => List.unmodifiable(_items);

  int get jumlahBelumDibeli =>
      _items.where((barang) => !barang.sudahDibeli).length;

  void tambah(
      String nama,
      int jumlah,
      String kategori,
      ) {
```




```dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

class Barang {
  String nama;
  int jumlah;
  String kategori;
  bool sudahDibeli;

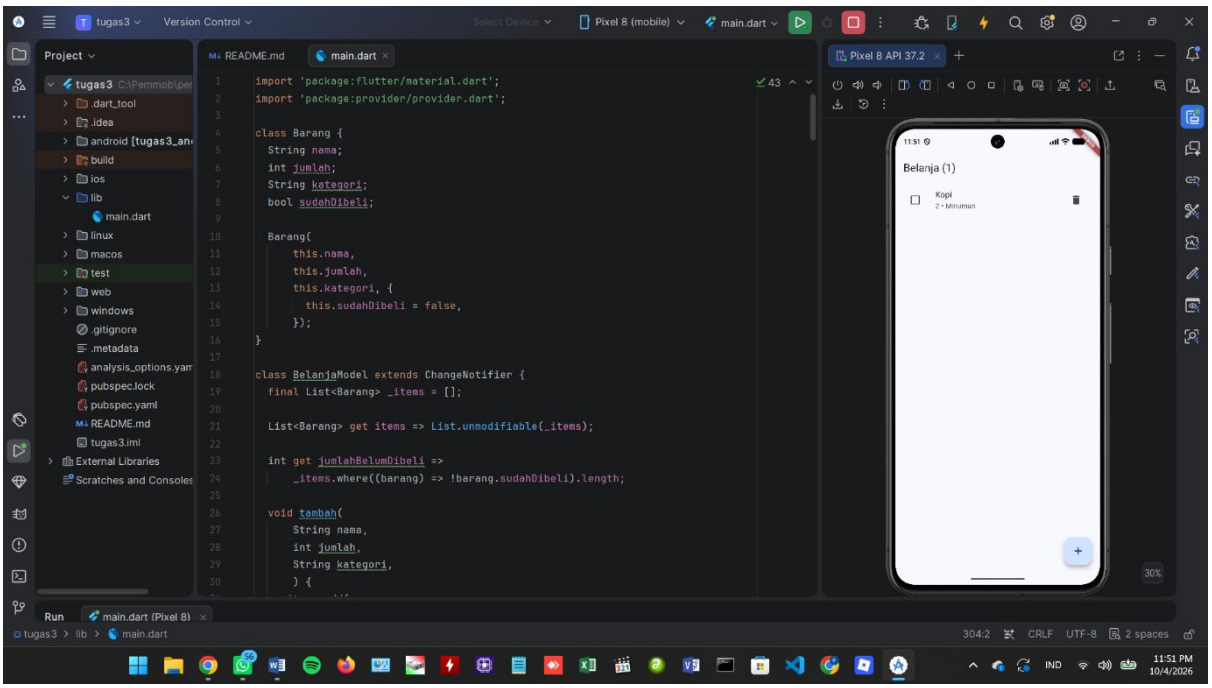
  Barang(
      this.nama,
      this.jumlah,
      this.kategori, {
        this.sudahDibeli = false,
      });
}

class BelanjaModel extends ChangeNotifier {
  final List<Barang> _items = [];

  List<Barang> get items => List.unmodifiable(_items);

  int get jumlahBelumDibeli =>
      _items.where((barang) => !barang.sudahDibeli).length;

  void tambah(
      String nama,
      int jumlah,
      String kategori,
      ) {
```



Pertanyaan Refleksi

1. Mengapa TextEditingController harus di-dispose?
   Jawab: TextEditingController harus di-dispose agar sumber daya yang digunakan dapat dilepaskan ketika widget sudah tidak digunakan. Jika tidak di-dispose, sumber daya bisa tetap terpakai dan berpotensi menyebabkan kebocoran memori (memory leak).
2. Kapan cukup memakai setState, dan kapan sebaiknya beralih ke Provider?
   Jawab: setState cukup digunakan jika perubahan data hanya memengaruhi satu widget atau bagian kecil dari tampilan. Provider lebih cocok digunakan jika data perlu dibagikan dan dikelola oleh beberapa widget agar pengelolaan state lebih terstruktur.
3. Apa yang terjadi bila notifyListeners() lupa dipanggil? Mengapa?
   Jawab: Tampilan yang menggunakan Provider tidak akan otomatis diperbarui meskipun nilai datanya sudah berubah. Hal ini terjadi karena notifyListeners() berfungsi memberi tahu widget yang mendengarkan perubahan bahwa data telah diperbarui.
4. Mengapa di dalam onPressed kita memakai context.read, bukan context.watch?
   Jawab: context.read digunakan untuk mengambil objek Provider tanpa berlangganan perubahan, sehingga cocok untuk menjalankan aksi seperti mengubah atau menyimpan data di dalam onPressed. Sementara itu, context.watch digunakan ketika widget perlu mendengarkan perubahan data dan membangun ulang tampilannya.